

Problem Set #7 — Graphs, Transpose/Inverse/Dual, Bron–Kerbosch

Aibek Zhazykbek

Overview

This report covers:

- Examples of directed graphs and their transposes.
- Examples of undirected graphs and their inverses (graph complements).
- Behaviour when the original graph is dense.
- Simple planar graphs and their duals; why dual is only defined for planar graphs.
- Example of a non-planar graph (with explanation why it has no dual).
- Bron–Kerbosch execution trace on the given small graph and list of maximal cliques.
- Python code for all operations and for drawing graphs.

1 Directed graphs and their transposes

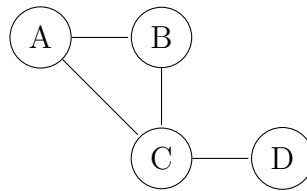
The *transpose* of a directed graph $G = (V, E)$ is $G^T = (V, E^T)$ where every edge $(u \rightarrow v) \in E$ is replaced by $(v \rightarrow u) \in E^T$.

Example 1 (small directed graph)

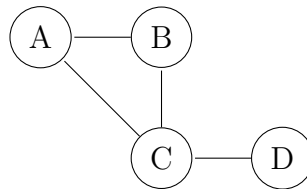
Vertices: A, B, C, D Edges:

$$A \rightarrow B, B \rightarrow C, A \rightarrow C, C \rightarrow D$$

Original (G):



Transpose (G^T):



Remark transpose is useful in e.g. Kosaraju's SCC algorithm.

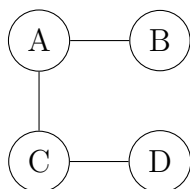
2 Undirected graphs and their inverse (complement)

The *inverse* (complement) $\overline{G}$ of an undirected simple graph $G = (V, E)$ has the same vertex set and edges $E' = \{\{u, v\} \mid u \neq v, \{u, v\} \notin E\}$. In other words all non-edges become edges and vice versa (excluding self-loops).

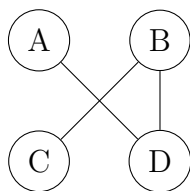
Example 2 (undirected)

Let $V = \{A, B, C, D\}$ and edges $E = \{\{A, B\}, \{A, C\}, \{C, D\}\}$.

Original (G):



Complement ($\overline{G}$):



What happens if original is dense?

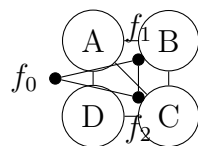
If G is dense (almost complete), then $\overline{G}$ is sparse (few edges). Complement swaps sparsity/density. For an extremely dense graph (complete), complement is edgeless.

3 Planar graphs and dual graphs

The *dual* of a planar graph G (embedded in the plane) is a graph G^* where each face of G (including the outer face) becomes a vertex in G^* , and two vertices of G^* are adjacent if the corresponding faces in G share an edge.

Example: square with one diagonal

Consider a planar embedding of a square with vertices A, B, C, D in that cyclic order and a diagonal AC . Faces: two triangular inner faces and one outer face.



This produces a triangular dual among faces f_0, f_1, f_2 .

Why dual only well-defined for planar graphs?

Dual depends on a planar embedding because faces are topological regions in the plane. Non-planar graphs lack a planar embedding without edge crossings, so faces are not defined consistently. Without faces you cannot place dual vertices or define adjacency by shared boundary edges.

Non-planar example: $K_{3,3}$ (utility graph) or K_5

Both $K_{3,3}$ and K_5 are non-planar (Kuratowski's theorem). They cannot be drawn without edge crossings; thus no planar faces, hence no dual.

4 Problem 2: Bron–Kerbosch on given graph

Graph G with vertices A, B, C, D , edges AB, AC, BC, CD . Adjacency list:

A: B, C
B: A, C
C: A, B, D
D: C

Initial call

BronKerbosch(R, P, X) initial: $R = \emptyset$, $P = \{A, B, C, D\}$, $X = \emptyset$.

We run Bron–Kerbosch (no pivot). We present the first recursive calls leading to a reported maximal clique:

Call 1: ($R=$, $P=A, B, C, D$, $X=$) Pick vertex A from P .

Call 2 (recurse with A): $R=A$, $P=B, C$, $X=$ (neighbors of A are B, C)
Pick B from P .

Call 3 (recurse with A,B): $R=A, B$, $P=C$, $X=$ (common neighbors of A and B is C) Pick C from P .

Call 4 (recurse with A,B,C): R=A,B,C, P=, X= Since P and X are empty, report R=A,B,C as a maximal clique.

This is the first maximal clique reported: $\{A, B, C\}$.

Continue with backtracking:

After reporting A,B,C, the algorithm backtracks, moves C from P to X in Call3, then returns etc. The other maximal cliques found will include C,D.

All maximal cliques

Listing all maximal cliques:

$$\{A, B, C\}, \quad \{C, D\}$$

Which are maximum? $\{A, B, C\}$ is maximum (size 3). $\{C, D\}$ is size 2.

Appendix: Python code

The following Python script implements:

- Directed transpose
- Undirected complement (inverse)
- Constructing dual for a given planar embedding (simple faces-based method)
- Bron–Kerbosch (with trace)
- Drawing graphs using networkx and saving images

```
# save this as exo7_graphs.py
import networkx as nx
from collections import deque
import matplotlib.pyplot as plt

# Directed transpose
def transpose_digraph(edges):
    """
    edges: list of tuples (u,v)
    returns list of edges (v,u)
    """
```

```

    """
    return [(v,u) for (u,v) in edges]

# Undirected complement
def complement_undirected(G):
    """
    G: networkx.Graph (undirected, simple)
    returns: new Graph that's complement of G
    """
    return nx.complement(G)

# Dual graph construction (for a given planar embedding
# )
def dual_from_planar_embedding(G, embedding):
    """
    G: networkx.Graph (planar)
    embedding: planar embedding data structure mapping
    edges to faces
    For simplicity this function expects that 'embedding
    ' is a list of faces,
    where each face is a list of vertices in cyclic
    order.
    Returns: Graph representing dual where nodes are
    faces (indexed).
    NOTE: This is a simple face-based dual construction
    for small examples.
    """
    # faces: list of lists (vertices in cyclic order)
    faces = embedding
    # Create dual graph
    D = nx.Graph()
    for i in range(len(faces)):
        D.add_node(i)
    # Two faces adjacent if they share an edge (
    # unordered pair of endpoints)
    def face_edges(face):
        edges = set()
        n = len(face)
        for i in range(n):
            a = face[i]

```

```

        b = face[(i+1)%n]
        edge = tuple(sorted((a,b)))
        edges.add(edge)
    return edges
face_edges_list = [face_edges(f) for f in faces]
for i in range(len(faces)):
    for j in range(i+1, len(faces)):
        if face_edges_list[i] & face_edges_list[j]:
            D.add_edge(i,j)
return D

# Bron--Kerbosch without pivot (trace)
def bron_kerbosch(R, P, X, neighbors, trace_list):
    """
    neighbors: dict vertex->set(neighbors)
    trace_list: list to append trace tuples (R,P,X) for
    debugging
    """
    trace_list.append((set(R), set(P), set(X)))
    if not P and not X:
        # report R as maximal clique
        return [set(R)]
    cliques = []
    for v in list(P):
        newR = set(R); newR.add(v)
        newP = set(P) & neighbors[v]
        newX = set(X) & neighbors[v]
        cliques += bron_kerbosch(newR, newP, newX,
                                neighbors, trace_list)
        P.remove(v)
        X.add(v)
    return cliques

#Drawing helper
def draw_graph(G, title='G', directed=False, filename=
None):
    plt.figure(figsize=(4,4))
    if directed:
        pos = nx.spring_layout(G, seed=42)
        nx.draw_networkx(G, pos=pos, with_labels=True,

```

```

        arrows=True)
    else:
        pos = nx.spring_layout(G, seed=42)
        nx.draw_networkx(G, pos=pos, with_labels=True)
    plt.title(title)
    plt.axis('off')
    if filename:
        plt.savefig(filename, bbox_inches='tight')
    plt.show()

# Example usage
if __name__ == "__main__":
    # Directed example
    edges = [('A','B'), ('B','C'), ('A','C'), ('C','D')]
    Gd = nx.DiGraph()
    Gd.add_edges_from(edges)
    draw_graph(Gd, title='Directed $\square$ G', directed=True,
        filename='directed_G.png')
    Gt_edges = transpose_digraph(edges)
    Gt = nx.DiGraph()
    Gt.add_edges_from(Gt_edges)
    draw_graph(Gt, title='Transpose $\square$ G $^T$ ', directed=True,
        filename='directed_Gt.png')

    # Undirected complement example
    Gu = nx.Graph()
    Gu.add_nodes_from(['A','B','C','D'])
    Gu.add_edges_from([('A','B'),('A','C'),('C','D')])
    draw_graph(Gu, title='Undirected $\square$ G', filename='
        undirected_G.png')
    Gc = complement_undirected(Gu)
    draw_graph(Gc, title='Complement $\square$ G $^c$ ', filename='
        undirected_Gc.png')

    # Planar dual example (square with diagonal AC)
    # Define embedding as face lists (outer face
    included)
    # Vertices positions: A(0,1),B(1,1),C(1,0),D(0,0)
    # Faces (cyclic vertex lists): two inner triangles (
    A,B,C) and (A,C,D) and outer face (A,D,C,B)

```

```

faces = [['A','B','C'], ['A','C','D'], ['A','D','C',
      'B']]
dual = dual_from_planar_embedding(Gu, faces)
draw_graph(dual, title='Dual (faces as nodes)',
      filename='dual.png')

# Bron-Kerbosch on given graph
Gbk = nx.Graph()
Gbk.add_nodes_from(['A','B','C','D'])
Gbk.add_edges_from([('A','B'),('A','C'),('B','C'),('C','D')])
neighbors = {v:set(Gbk.neighbors(v)) for v in Gbk.nodes()}
trace = []
cliques = bron_kerbosch(set(), set(Gbk.nodes()), set(), neighbors, trace)
print("Trace (first few calls):")
for t in trace[:8]:
    print("R=",t[0], "P=",t[1], "X=",t[2])
print("Maximal cliques:", cliques)
draw_graph(Gbk, title='Graph for Bron-Kerbosch',
      filename='bk_graph.png')

```